Estructura del proyecto

Backend de Dietetica - guia de carpetas, responsabilidades y flujo

Vista general

El backend recibe solicitudes HTTP, valida autenticacion y permisos, ejecuta la logica de negocio y consulta o modifica la base SQLite.

Recorrido

Cliente (Postman o frontend) -> routes/ -> middleware (si la ruta es privada) -> services/ -> models/ -> dietetica.db.

Stack

Node.js ejecuta JavaScript; Express maneja rutas HTTP; Sequelize conecta los modelos con SQLite; JWT autentica las solicitudes.

Carpetas principales

src/

Codigo de la aplicacion: inicio del servidor, configuracion, conexion a base, modelos, servicios y middleware.

routes/

Define los endpoints y conecta cada metodo HTTP con el servicio correspondiente.

node_modules/

Dependencias instaladas por npm. No se edita a mano ni se sube a Git.

Archivos de la raiz

package.json / package-lock.json

Dependencias del backend y versiones instaladas; tambien define comandos como npm run dev y npm start.

.env / .env.example

Configuracion local (puerto, origen frontend y JWT_SECRET). El archivo .env contiene secretos y no se comparte.

dietetica.db

Base de datos SQLite usada por el backend.

*.sql

Esquemas y scripts de migracion de la base; verificar y respaldar la base antes de ejecutar una migracion.

Dentro de src/

Configuracion, autenticacion, reglas de negocio y modelos de datos

Configuracion y arranque

src/index.js

Crea la app Express, instala middleware comun, monta las rutas, conecta con SQLite e inicia el servidor.

src/config.js

Lee y valida configuracion como PORT, FRONTEND_ORIGIN y JWT_SECRET.

src/db.js

Configura Sequelize y la conexion al archivo dietetica.db.

Modelos: src/models/

Usuario.js

Representa la tabla Usuarios: email, hash de contrasena, rol y estado.

Producto.js

Representa Productos: nombre, precio, stock y otros datos del catalogo.

Ticket.js

Representa la cabecera de una compra: usuario, fecha y monto total.

DetalleTicket.js

Representa las lineas de Tickets_Productos: producto, cantidad e importe.

associations.js

Define relaciones entre usuarios, tickets, detalles y productos.

Servicios: src/services/

auth.service.js

Registra clientes, comprueba credenciales, guarda contrasenas con bcrypt y emite JWT.

user.service.js

Lista y administra usuarios; el administrador puede crear cuentas y cambiar rol o estado.

product.service.js

Consulta y realiza ABM de productos, con validacion de los datos recibidos.

ticket.service.js

Crea y consulta compras, calcula importes y trabaja con transacciones para conservar consistencia.

Middleware: src/middleware/

auth.js

Lee Authorization: Bearer, verifica firma y vencimiento del JWT, confirma cuenta/rol vigentes y aplica permisos.

Rutas, roles y uso

Como se conectan los endpoints y como probarlos desde Postman

Rutas: routes/

auth.routes.js

POST /api/auth/register y POST /api/auth/login son publicos; GET /api/auth/me requiere JWT.

users.routes.js

GET y POST /api/Usuarios, y PATCH /api/Usuarios/:id: operaciones de administracion de usuarios.

products.routes.js

GET /api/Productos y operaciones POST, PUT y DELETE para administrar el catalogo.

tickets.routes.js

GET y POST /api/Tickets, mas GET /api/Tickets/:id y PUT /api/Tickets/:id.

Permisos por rol

Administrador

Lista y crea usuarios, cambia sus roles/estados y puede editar tickets.

Vendedor

Consulta, crea, edita y elimina productos.

Cliente

Consulta productos, registra una compra y solo consulta sus propios tickets. El backend toma su id desde el JWT.

Respuestas

401 significa que falta un token valido; 403 indica que el usuario autenticado no tiene permiso.

Flujo de prueba con Postman

1. Registro

POST /api/auth/register con nombre, email y password. El registro publico asigna siempre el rol Cliente.

2. Login

POST /api/auth/login con email y password; la respuesta incluye un token con vencimiento de una hora.

3. Ruta protegida

En Authorization elige Bearer Token y pega el token. Prueba GET /api/auth/me o GET /api/Productos.

4. Compra

POST /api/Tickets con Authorization Bearer y un body con productos e id_producto/cantidad. No enviar id_usuario: se obtiene del token.